The MiniJava Type System

CS3300 course revision — 7 September 2026

Based on Jens Palsberg's *The MiniJava Type System*, October 2011

This course revision incorporates the Assignment 2 clarifications into the rules themselves. Rule (16) permits covariant return types in overriding; rule (21) permits a return expression to have a subtype of the declared return type. Section 4 explicitly includes reflexivity for all MiniJava types. Rule (15) remains unchanged. Original section and rule numbers are retained; rule (16) is split into (16a) and (16b). Method types consistently omit parameter names.

1 What is MiniJava?

MiniJava is a small Java-like language with classes, single inheritance, methods, integer arrays, and basic statements and expressions. Method overloading is not allowed.

The statement `System.out.println(...);` prints integers, and `e.length` applies to an expression of type `int[]`.

2 Syntax

The core syntax below follows the original reference. The course MiniJava grammar specifies the complete syntax accepted in Assignment 2, including its additional operators and statement forms. Nonterminals are italic, keywords are in typewriter font, and lists written with dots may be empty. Superscripts and subscripts distinguish metavariables.

$$
\begin{array}{rrl}
\textit{Goal} & g & ::= mc\ d_1 \ldots d_n \\
\textit{MainClass} & mc & ::= \texttt{class}\ id\ \{\texttt{public static void main (String[]}\ id^S\texttt{)\{} \\
 & & \quad t_1\ id_1; \ldots; t_r\ id_r;\ s_1 \ldots s_q\ \}\} \\
\textit{TypeDeclaration} & d & ::= \texttt{class}\ id\ \{t_1\ id_1; \ldots; t_f\ id_f;\ m_1 \ldots m_k\} \\
 & & \quad \mid \texttt{class}\ id\ \texttt{extends}\ id^P\ \{t_1\ id_1; \ldots; t_f\ id_f;\ m_1 \ldots m_k\} \\
\textit{MethodDeclaration} & m & ::= \texttt{public}\ t\ id^M(t_1^F\ id_1^F, \ldots, t_n^F\ id_n^F)\ \{ \\
 & & \quad t_1\ id_1; \ldots; t_r\ id_r;\ s_1 \ldots s_q\ \texttt{return}\ e;\ \} \\
\textit{Type} & t & ::= \texttt{int[]} \mid \texttt{boolean} \mid \texttt{int} \mid id \\
\textit{Statement} & s & ::= \{s_1 \ldots s_q\} \mid id = e; \mid id[e_1] = e_2; \\
 & & \quad \mid \texttt{if}\ (e)\ s_1\ \texttt{else}\ s_2 \mid \texttt{while}\ (e)\ s \\
 & & \quad \mid \texttt{System.out.println}(e); \\
\textit{Expression} & e & ::= p_1\ \texttt{\&\&}\ p_2 \mid p_1 < p_2 \mid p_1 + p_2 \mid p_1 - p_2 \mid p_1 * p_2 \\
 & & \quad \mid p_1[p_2] \mid p.\texttt{length} \mid p.id(e_1, \ldots, e_n) \mid p \\
\textit{PrimaryExpression} & p & ::= c \mid \texttt{true} \mid \texttt{false} \mid id \mid \texttt{this} \\
 & & \quad \mid \texttt{new int}[e] \mid \texttt{new}\ id() \mid !e \mid (e) \\
\textit{IntegerLiteral} & c & ::= \langle \textsc{integer literal} \rangle \\
\textit{Identifier} & id & ::= \langle \textsc{identifier} \rangle
\end{array}
$$

In a type position, *id* must name a declared class: for example, `Animal` in `Animal pet;` is a class type.

3 Notation for Rules

An inference rule has the form

$$\frac{\mathit{hypothesis}_1 \quad \mathit{hypothesis}_2 \quad \cdots \quad \mathit{hypothesis}_n}{\mathit{conclusion}}.$$

It states that the conclusion follows whenever every hypothesis holds. A rule with no hypotheses is an axiom; its horizontal bar may be omitted. A derivation is a tree whose leaves are axioms and whose internal nodes apply inference rules. The root is the judgment established by the derivation.

4 Subtyping

We write $t_1 \leq t_2$ when t_1 is a subtype of t_2. The relation is reflexive on *all* MiniJava types. On class names, it is the reflexive, transitive closure of the declared `extends` relation.

$$t \leq t \tag{1}$$

$$\frac{t_1 \leq t_2 \qquad t_2 \leq t_3}{t_1 \leq t_3} \tag{2}$$

$$\frac{\texttt{class}\ C\ \texttt{extends}\ D\ \ldots \text{ is in the program}}{C \leq D} \tag{3}$$

In particular, rule (1) includes

$$\texttt{int} \leq \texttt{int}, \qquad \texttt{boolean} \leq \texttt{boolean}, \qquad \texttt{int[]} \leq \texttt{int[]}.$$

There are no subtype relationships between these three types, or between any of these types and a class type. Thus the only subtype of each of `int`, `boolean`, and `int[]` is itself.

5 Type Environments

A type environment A is a finite mapping from identifiers to types; its domain is written $\mathrm{dom}(A)$. For pairwise distinct identifiers, $[id_1 : t_1, \ldots, id_r : t_r]$ maps id_i to t_i for $i \in 1..r$. Environment composition gives the right-hand environment precedence:

$$(A_1 \cdot A_2)(id) = \begin{cases} A_2(id) & \text{if } id \in \mathrm{dom}(A_2), \\ A_1(id) & \text{otherwise.} \end{cases} \tag{4}$$

6 Helper Functions

6.1 The *classname* Helper Function

The function classname returns the name of a class declaration.

$$\text{classname}(\texttt{class}\ id\ \{\texttt{public static void main(String[]}\ id^S)\{\ldots\}\}) = id \tag{5}$$

$$\text{classname}(\texttt{class}\ id\ \{t_1\ id_1;\ldots;t_f\ id_f;\ m_1\ldots m_k\}) = id \tag{6}$$

$$\text{classname}(\texttt{class}\ id\ \texttt{extends}\ id^P\ \{t_1\ id_1;\ldots;t_f\ id_f;\ m_1\ldots m_k\}) = id \tag{7}$$

6.2 The *methodname* Helper Function

The function methodname returns the name of a method declaration.

$$\text{methodname}(\texttt{public}\ t\ id^M(\ldots)\ \ldots) = id^M \tag{8}$$

6.3 The *distinct* Helper Function

The predicate distinct checks that identifiers in a list are pairwise distinct.

$$\frac{\forall i \in 1..n:\ \forall j \in 1..n:\ id_i = id_j \Rightarrow i = j}{\text{distinct}(id_1, \ldots, id_n)} \tag{9}$$

6.4 The *fields* Helper Function

The environment fields(C) contains the fields of C and its superclasses. Fields declared in C take precedence over inherited fields of the same name.

$$\frac{\texttt{class}\ id\ \{t_1\ id_1;\ldots;t_f\ id_f;\ m_1\ldots m_k\} \text{ is in the program}}{\text{fields}(id) = [id_1 : t_1, \ldots, id_f : t_f]} \tag{10}$$

$$\frac{\texttt{class}\ id\ \texttt{extends}\ id^P\ \{t_1\ id_1;\ldots;t_f\ id_f;\ m_1\ldots m_k\} \text{ is in the program}}{\text{fields}(id) = \text{fields}(id^P) \cdot [id_1 : t_1, \ldots, id_f : t_f]} \tag{11}$$

6.5 The *methodtype* Helper Function

The value methodtype(id, id^M) is the ordered list of parameter types and return type of method id^M in class id, including inherited methods. It is $\perp$ if no such method exists. A method type has the form $(t_1, \ldots, t_n) \to t$. Parameter names are not part of the method type.

$$\frac{\begin{array}{c}\texttt{class } id\ \{\ldots\ m_1 \ldots m_k\} \text{ is in the program} \\ \text{for some } j \in 1..k:\ \text{methodname}(m_j) = id^M \\ m_j \text{ is of the form } \texttt{public } t\ id^M(t_1^F\ id_1^F, \ldots, t_n^F\ id_n^F)\ \{t_1\ id_1; \ldots; t_r\ id_r;\ s_1 \ldots s_q\ \texttt{return } e;\}\end{array}}{\text{methodtype}(id, id^M) = (t_1^F, \ldots, t_n^F) \to t} \tag{12}$$

$$\frac{\begin{array}{c}\texttt{class } id\ \{\ldots\ m_1 \ldots m_k\} \text{ is in the program} \\ \text{for all } j \in 1..k:\ \text{methodname}(m_j) \neq id^M\end{array}}{\text{methodtype}(id, id^M) = \perp} \tag{13}$$

$$\frac{\begin{array}{c}\texttt{class } id\ \texttt{extends } id^P\ \{\ldots\ m_1 \ldots m_k\} \text{ is in the program} \\ \text{for some } j \in 1..k:\ \text{methodname}(m_j) = id^M \\ m_j \text{ is of the form } \texttt{public } t\ id^M(t_1^F\ id_1^F, \ldots, t_n^F\ id_n^F)\ \{t_1\ id_1; \ldots; t_r\ id_r;\ s_1 \ldots s_q\ \texttt{return } e;\}\end{array}}{\text{methodtype}(id, id^M) = (t_1^F, \ldots, t_n^F) \to t} \tag{14}$$

$$\frac{\begin{array}{c}\texttt{class } id\ \texttt{extends } id^P\ \{\ldots\ m_1 \ldots m_k\} \text{ is in the program} \\ \text{for all } j \in 1..k:\ \text{methodname}(m_j) \neq id^M\end{array}}{\text{methodtype}(id, id^M) = \text{methodtype}(id^P, id^M)} \tag{15}$$

Rule (15) is an inheritance lookup rule: it applies only when the subclass does not declare the method. Its equality is unchanged.

6.6 The *noOverloading* Helper Function

For a method declared in class id with direct superclass id^P, either there is no inherited method of that name, or the declaration must be a valid override. An override has exactly the same parameter types, in the same order, and a return type that is the same as, or a subtype of, the inherited return type.

$$\frac{\text{methodtype}(id^P, id^M) = \perp}{\text{noOverloading}(id, id^P, id^M)} \tag{16a}$$

$$\frac{\begin{array}{c}\text{methodtype}(id, id^M) = (t_1, \ldots, t_n) \to t \\ \text{methodtype}(id^P, id^M) = (t'_1, \ldots, t'_n) \to t' \\ t_i = t'_i \quad (i \in 1..n) \qquad t \leq t'\end{array}}{\text{noOverloading}(id, id^P, id^M)} \tag{16b}$$

Both parameter lists have length n; parameter names may differ. Changing a parameter type, even to a subtype, or changing the parameter count is forbidden.

7 Type Rules

7.1 Type Judgments

We use the following seven forms of judgments:

Judgment	Meaning
$\vdash g$	The goal (whole program) type checks.
$\vdash mc$	The main class type checks.
$\vdash d$	The type declaration type checks.
$C \vdash m$	The method declaration type checks in class C.
$A, C \vdash s$	The statement type checks in environment A, class C.
$A, C \vdash e : t$	The expression has type t in environment A, class C.
$A, C \vdash p : t$	The primary expression has type t in that context.

7.2 Goal

$$\frac{\text{distinct}(\text{classname}(mc), \text{classname}(d_1), \ldots, \text{classname}(d_n)) \qquad \vdash mc \qquad \vdash d_i \quad (i \in 1..n)}{\vdash mc\ d_1 \ldots d_n} \tag{17}$$

7.3 Main Class

$$\frac{\text{distinct}(id_1, \ldots, id_r) \qquad [id_1 : t_1, \ldots, id_r : t_r], \bot \vdash s_i \quad (i \in 1..q)}{\vdash \texttt{class}\ id\ \{\texttt{public static void main(String[]}\ id^S\texttt{)}\{t_1\ id_1; \ldots; t_r\ id_r;\ s_1 \ldots s_q\}\}} \tag{18}$$

Here $\bot$ in the class position denotes the absence of an instance context; `this` is unavailable in the static main method.

Examples of the corrected return and overriding rules

Assume `B extends A`. Each row considers the stated type constraint; the rest of the program must also type check.

Situation	Result
A method declared to return `A` returns `new B()`.	Allowed
A method declared to return `B` returns `new A()`.	Type error
An inherited `A f(A x)` is overridden by `B f(A y)`.	Allowed
An inherited `A f(A x)` is replaced by `B f(B x)`.	Type error: parameter type changed
An inherited `B f(A x)` is overridden by `A f(A x)`.	Type error: return type widened

The parameter equality in rule (16b) concerns method *declarations*. At a method *call*, rule (35) permits argument expressions whose types are subtypes of the declared parameter types.

7.4 Type Declarations

$$\frac{\begin{array}{c}\text{distinct}(id_1, \ldots, id_f) \\ \text{distinct}(\text{methodname}(m_1), \ldots, \text{methodname}(m_k)) \\ id \vdash m_i \quad (i \in 1..k)\end{array}}{\vdash \texttt{class}\ id\ \{t_1\ id_1; \ldots; t_f\ id_f;\ m_1 \ldots m_k\}} \tag{19}$$

$$\frac{\begin{array}{c}\text{distinct}(id_1, \ldots, id_f) \\ \text{distinct}(\text{methodname}(m_1), \ldots, \text{methodname}(m_k)) \\ \text{noOverloading}(id, id^P, \text{methodname}(m_i)) \qquad id \vdash m_i \quad (i \in 1..k)\end{array}}{\vdash \texttt{class}\ id\ \texttt{extends}\ id^P\ \{t_1\ id_1; \ldots; t_f\ id_f;\ m_1 \ldots m_k\}} \tag{20}$$

7.5 Method Declarations

$$\frac{\begin{array}{c}\text{distinct}(id_1^F, \ldots, id_n^F) \qquad \text{distinct}(id_1, \ldots, id_r) \\ A = \text{fields}(C) \cdot [id_1^F : t_1^F, \ldots, id_n^F : t_n^F] \cdot [id_1 : t_1, \ldots, id_r : t_r] \\ A, C \vdash s_i \quad (i \in 1..q) \qquad A, C \vdash e : t' \qquad t' \leq t\end{array}}{C \vdash \texttt{public}\ t\ id^M(t_1^F\ id_1^F, \ldots, t_n^F\ id_n^F)\ \{t_1\ id_1; \ldots; t_r\ id_r;\ s_1 \ldots s_q\ \texttt{return}\ e;\}} \tag{21}$$

The return expression's type t' may be a subtype of the declared return type t. This check applies to every method, whether or not it overrides another.

7.6 Statements

$$\frac{A, C \vdash s_i \quad (i \in 1..q)}{A, C \vdash \{s_1 \ldots s_q\}} \tag{22}$$

$$\frac{A(id) = t_1 \qquad A, C \vdash e : t_2 \qquad t_2 \leq t_1}{A, C \vdash id = e;} \tag{23}$$

$$\frac{A(id) = \texttt{int[]} \qquad A, C \vdash e_1 : \texttt{int} \qquad A, C \vdash e_2 : \texttt{int}}{A, C \vdash id[e_1] = e_2;} \tag{24}$$

$$\frac{A, C \vdash e : \texttt{boolean} \qquad A, C \vdash s_1 \qquad A, C \vdash s_2}{A, C \vdash \texttt{if}\ (e)\ s_1\ \texttt{else}\ s_2} \tag{25}$$

$$\frac{A, C \vdash e : \texttt{boolean} \qquad A, C \vdash s}{A, C \vdash \texttt{while}\ (e)\ s} \tag{26}$$

$$\frac{A, C \vdash e : \texttt{int}}{A, C \vdash \texttt{System.out.println}(e);} \tag{27}$$

7.7 Expressions and Primary Expressions

$$\frac{A, C \vdash p_1 : \texttt{boolean} \qquad A, C \vdash p_2 : \texttt{boolean}}{A, C \vdash p_1 \texttt{ \&\& } p_2 : \texttt{boolean}} \tag{28}$$

$$\frac{A, C \vdash p_1 : \texttt{int} \qquad A, C \vdash p_2 : \texttt{int}}{A, C \vdash p_1 < p_2 : \texttt{boolean}} \tag{29}$$

$$\frac{A, C \vdash p_1 : \texttt{int} \qquad A, C \vdash p_2 : \texttt{int}}{A, C \vdash p_1 + p_2 : \texttt{int}} \tag{30}$$

$$\frac{A, C \vdash p_1 : \texttt{int} \qquad A, C \vdash p_2 : \texttt{int}}{A, C \vdash p_1 - p_2 : \texttt{int}} \tag{31}$$

$$\frac{A, C \vdash p_1 : \texttt{int} \qquad A, C \vdash p_2 : \texttt{int}}{A, C \vdash p_1 * p_2 : \texttt{int}} \tag{32}$$

$$\frac{A, C \vdash p_1 : \texttt{int[]} \qquad A, C \vdash p_2 : \texttt{int}}{A, C \vdash p_1[p_2] : \texttt{int}} \tag{33}$$

$$\frac{A, C \vdash p : \texttt{int[]}}{A, C \vdash p.\texttt{length} : \texttt{int}} \tag{34}$$

$$\frac{\begin{array}{c} A, C \vdash p : D \qquad \text{methodtype}(D, id) = (t'_1, \ldots, t'_n) \to t \\ A, C \vdash e_i : t_i \qquad t_i \leq t'_i \quad (i \in 1..n) \end{array}}{A, C \vdash p.id(e_1, \ldots, e_n) : t} \tag{35}$$

$$A, C \vdash c : \texttt{int} \tag{36}$$

$$A, C \vdash \texttt{true} : \texttt{boolean} \tag{37}$$

$$A, C \vdash \texttt{false} : \texttt{boolean} \tag{38}$$

$$\frac{id \in \text{dom}(A)}{A, C \vdash id : A(id)} \tag{39}$$

$$\frac{C \neq \bot}{A, C \vdash \texttt{this} : C} \tag{40}$$

$$\frac{A, C \vdash e : \texttt{int}}{A, C \vdash \texttt{new int}[e] : \texttt{int[]}} \tag{41}$$

$$A, C \vdash \texttt{new } id() : id \tag{42}$$

$$\frac{A, C \vdash e : \texttt{boolean}}{A, C \vdash !e : \texttt{boolean}} \tag{43}$$

$$\frac{A, C \vdash e : t}{A, C \vdash (e) : t} \tag{44}$$